

Towards a Trusted Third Party (in a Box)

Enabling Third-Party Auditing and Verifiably-Scoped Monitoring with an LLM in a Trusted Execution Environment

Abstract:

We built a *trusted-third-party-in-a-box*: an open-source LLM, running inside a trusted execution environment (TEE), that executes commands two parties have agreed on over private data that not all of them can see.

We decompose the problem of building a robust trusted-third-party-in-a-box into two kinds. *Technical* problems are specific to a given implementation and trust level e.g. how do you trust the hardware to run the code you claim? *Process* problems are the ones that persist across every level of trust. We focus on the process problems, because that's where you can make incremental, broadly-useful progress.

A protocol like this has many applications. Most centrally,, a monitor can prove it's checking for exactly what it claims, and nothing else. We develop two near-term applications: verifiably-scoped monitoring, a relaxation of zero-data-retention we call ZDR-lite; and recurring third-party auditing. We release an open-source implementation that runs in a real TEE, with a live demo at <https://auditor-in-a-box.royrinberg.com/>, and we'd love for people to poke at it.

1. General Problem: Secure Multi-party computation

Two firms may want to know whether a merger creates a conflict of interest; a lab may want to monitor its users for misuse without reading their conversations; an auditor might want to confirm a company *isn't* doing something without being handed its source code. These cases all illustrate an extremely common need – to compute over data we can't fully share with others.

Cryptography has a name for this: **secure multi-party computation (MPC)**, which lets mutually distrustful parties jointly evaluate a function over their private inputs while learning nothing beyond the result. The problem was introduced by Andrew Yao in 1982 — the "Millionaires' Problem," two people learning who is richer without revealing their wealth *Any function can in principle be computed through MPC*. However, in practice, the overhead associated with MPC can be prohibitively large (in both computation and communication complexity), and the usability can be difficult because any computation must be specified formally.

While cryptography formalizes the problem, and offers many of its own solutions, society has developed an older, lower-tech solution: relying on a trusted human intermediary. Two firms hire lawyers to read both parties' financial documents and report only partial information (e.g., whether a conflict of interest exists). The lawyers are given access to all the company's documents, and their credibility relies on their ability to preserve client confidentiality and provide reliable counsel. . The intermediary handles precisely the messy, natural-language, judgment-laden tasks that formal MPC cannot — at the cost of having to trust them.

This blog post and line of work is about replacing that intermediary with a trusted LLM in a trusted box. This idea has been floating around in the [ecosystem for a while now](#). And recent work on [Trusted Capable Model Environments](#) argues that a capable model can fill the trusted-third-party role, enabling secure computations that were previously infeasible.

There are many different settings where you may want a trusted third party: two acquaintances checking something casually; a company and an enterprise customer; one lab verifying another; two countries verifying each other under a treaty. Different settings place different demands on such a scheme; in particular, they change *how much you trust each component*: how much you trust the hardware to run the code you think it's running, how hard you expect the other side to try to exploit the model, how much you care about privacy.

That suggests splitting the problems in two: **technical** problems, specific to a given implementation and trust level (*how do I secure this network against this particular adversary?*), and **process** problems, the ones that persist across every level of trust (*how do two parties agree on what the box should do, and what happens when it's wrong?*). The same process problems show up whether it's METR auditing a friendly lab, or two distrustful nation-states.

We develop two near-term applications of the trusted-third-party-in-a-box: **verifiably-scoped monitoring**, a relaxation of zero-data-retention we call **ZDR-lite** that lets a lab do safety monitoring while bounding what it's allowed to monitor *for*; and **recurring third-party auditing**, where an external auditor makes repeated, verifiable checks on a lab's internal practice.

2. The primitive: a trusted LLM in a trusted box

The core method here is simple: we run an open-source LLM inside a trusted execution environment (TEE). A **Trusted Execution Environment** (TEE) is a hardware-isolated region of a processor that provides two guarantees:

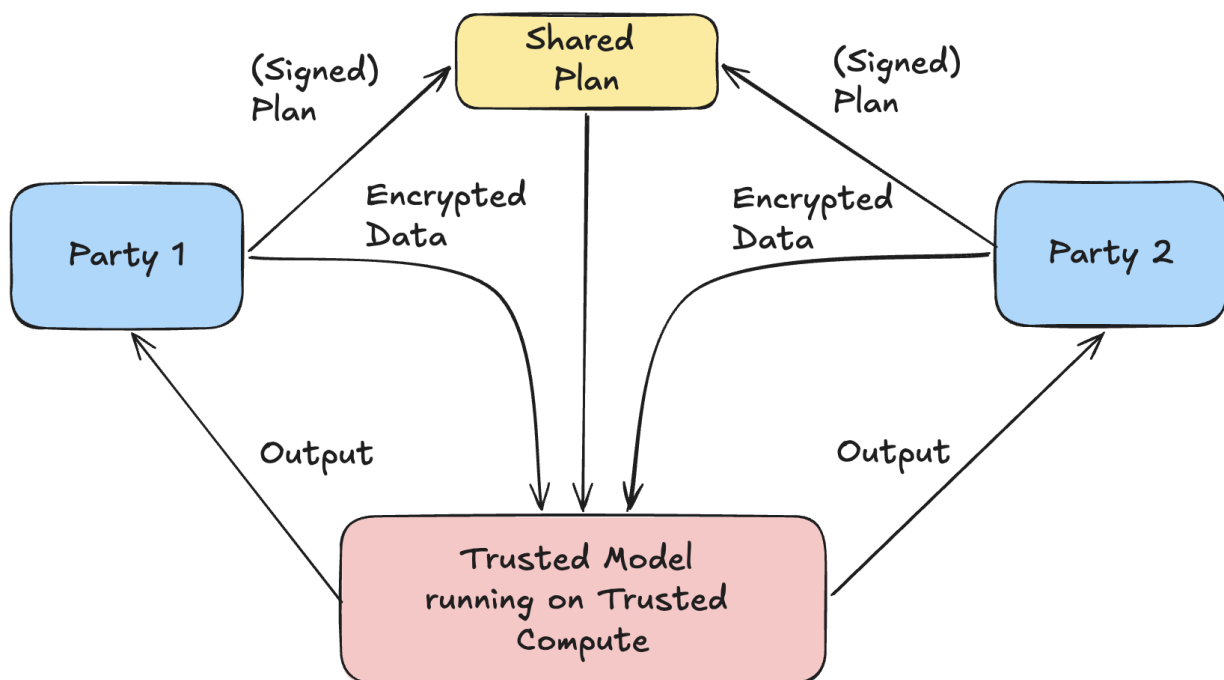
- **Confidentiality**: enclave memory is not readable by the host operating system, the hypervisor, or the infrastructure provider.
- **Integrity**: the binary loaded into the enclave is measured at launch, the measurement is signed by the hardware, and remote attestation allows an external party to verify that a specific binary is running.

Any modification to the executing code changes the measurement and is detectable. Together, these two guarantees give you that: the enclave runs exactly the attested code and nothing else, and nothing inside it is visible from outside.

As a consequence, if the signed code does not release a given piece of information, that information never leaves the box, and the data being released would require running different code, which would change the launch measurement and fail attestation. The TEE thus converts a promise ("we handled your data as agreed") into a verifiable statement ("this exact code, which you can audit, ran").

The TEE guarantees faithful execution, but it's agnostic about *what* gets executed. So the box is only useful once the two sides have settled, in advance, on exactly what it should do. We refer to this specification as a **Plan**, jointly signed by both parties, which specifies the inputs, the computation steps (e.g. python code and LLM calls) to be executed, and the outputs to be released. Any change of scope would require additional agreement from both parties.

We outline the overall flow below. Both parties agree on a plan which they both sign and send to the TEE. Then they send private data to the TEE directly, which is encrypted with the public key of the TEE. Both parties receive whatever output they previously agreed on.



There are 4 main desiderata:

- **Correctness.** The released outputs are a faithful application of the signed plan to the submitted inputs. We use the TEE's attestation to ensure the plan binary is run and nothing else is run to achieve Correctness.
- **Confinement.** Computation outside the plan is never performed; e.g. out-of-scope queries are rejected. We make sure only the agreed upon plan runs using the TEE's attestation and we use the open-source LLM as a scope-checker (e.g. to reject monitoring queries that are out-of-scope) to achieve Confinement.
- **Data confidentiality.** Each party learns only the released outputs, not the other party's inputs or any intermediate state outside the agreed scope. We achieve this by having each party's data encrypted with the TEE's public key such that only the TEE can decrypt the data and by filtering the outputs as part of the plan.

3. Process-level problems

We flag three, top-level process problems that need to be resolved for (nearly?) any third party auditing mechanism.

3.1 Plan negotiation

Concretely, we think of a plan as a computation graph built of nodes for inputs, computation steps, and outputs. Both the structure of the graph and each individual node are agreed to and signed. How do you agree on the exact program that's going to run in the box (e.g. the prompts, the code)? If you're prompting an LLM, there are many ways to write any given prompt, and both parties need to be confident that the process they *believe* is going to run, is the process that actually runs.

The act of agreeing on a structure/prompt/constitution is quite similar to existing procedures for writing protocols: one party drafts it, the other reviews and edits it, both parties iterate and negotiate. While this is not novel, it is also not trivial, and requires thought and effort. In the same way that writing legal contracts generally requires lawyers, and writing smart contracts requires specialized developers, these kinds of compute contracts need good tooling.

What we imagine we'd need is mostly a better UI, something like a Google Docs for plan-writing. It is important for the plan negotiation to stay legible. The vocabulary has to stay small enough that a non-specialist can read a plan and be confident about what it will and won't do. This is the primary reason for holding a plan to just two node types and three stages: it keeps the surface small enough that review is tractable, in the same way a well-structured contract is.

3.2 False positives and appeals

Problem two: what happens when the auditor in the box is wrong? We have asked the LLM to check something, and it flags something by mistake. How does one party credibly convince the other party it was a mistake and not a true positive?

To think about this part, it's easiest to think in the context of classifiers, where a classifier only has 4 states: True Positive, True Negative, False Positive, and False Negative. While we can imagine doing other forms of computation in such a box, it's easiest to reason about classifiers, and the logic generally transfers. Let's be precise about what a false positive is:

- a *positive* means the classifier in the box said something was dangerous;
- a *false positive* means it said dangerous when in fact it wasn't.

The key observation is that a false positive, by definition, means there's *exonerating evidence* somewhere — if the flag is genuinely wrong, then some piece of context exists that would show it's wrong. So the flagged party should be able to provide that evidence, or more context: "the prompt thought I was doing this, but actually this other thing was happening — here's a new question that you'll agree is of the same flavor, and its answer will confirm what I'm saying."

This observation immediately provides us with the structure for an appeals process. When a query is flagged, the responsible party is given some time window to supply exonerating evidence that could be of one of two forms:

1. **A minimal data modification** that removes the flag — for example, correcting a typographical artifact in the underlying data that triggered the classifier, after which re-execution no longer triggers.
2. **A verifiable secondary query** under a different prompt that demonstrates the original flag was spurious — for example, establishing that a flagged reference to a public figure is in fact an internal product name.

Both forms get evaluated *inside* the box under signed prompts, so the appeal can't be used by the monitored party to sneak data out, or by the monitor to quietly widen the scope. The burden of resolution is shifted onto the party that triggered the flag. That party has exonerating evidence exactly when the flag was spurious, and doesn't when it wasn't.

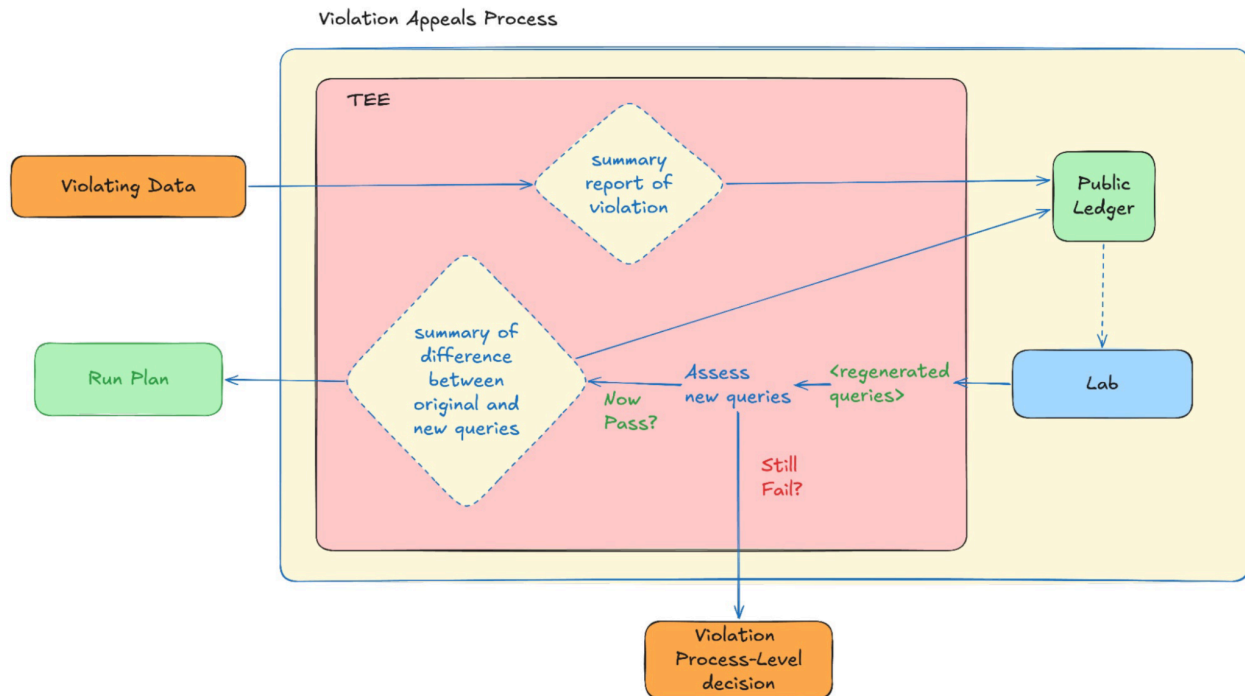


Figure: Here we see a diagram for how a lab can appeal a setting where one of their queries triggers a false positive flag. We write in the appendix for how to handle dealing with people trying to abuse the [false positive appeals process](#).

By leaning on incentive alignment and making the loop interactive we are able to resolve basically any dispute: a wrongly-flagged party is motivated to produce the evidence that clears them.

We note that because we have a clean way to handle false positives, we can afford to make the classifier *more* sensitive, and bias it toward false positives to avoid false negatives.

3.3 Out-of-distribution inputs and prompt injection

The third problem is that the data going into the box is private, so there's always a risk the model does something weird on input that's far out of distribution (the adversarial version of out-of-distribution input is a prompt injection).

However, to prompt-inject a model, you generally have to *try*: you can't reliably pull it off on the first attempt (as made particularly explicit in [this paper](#)). One major difficulty of the system we are proposing is that the LLM (in the box) is open source, so a party can poke the LLM on their own cluster and arrive at a prompt injection they're confident will work.

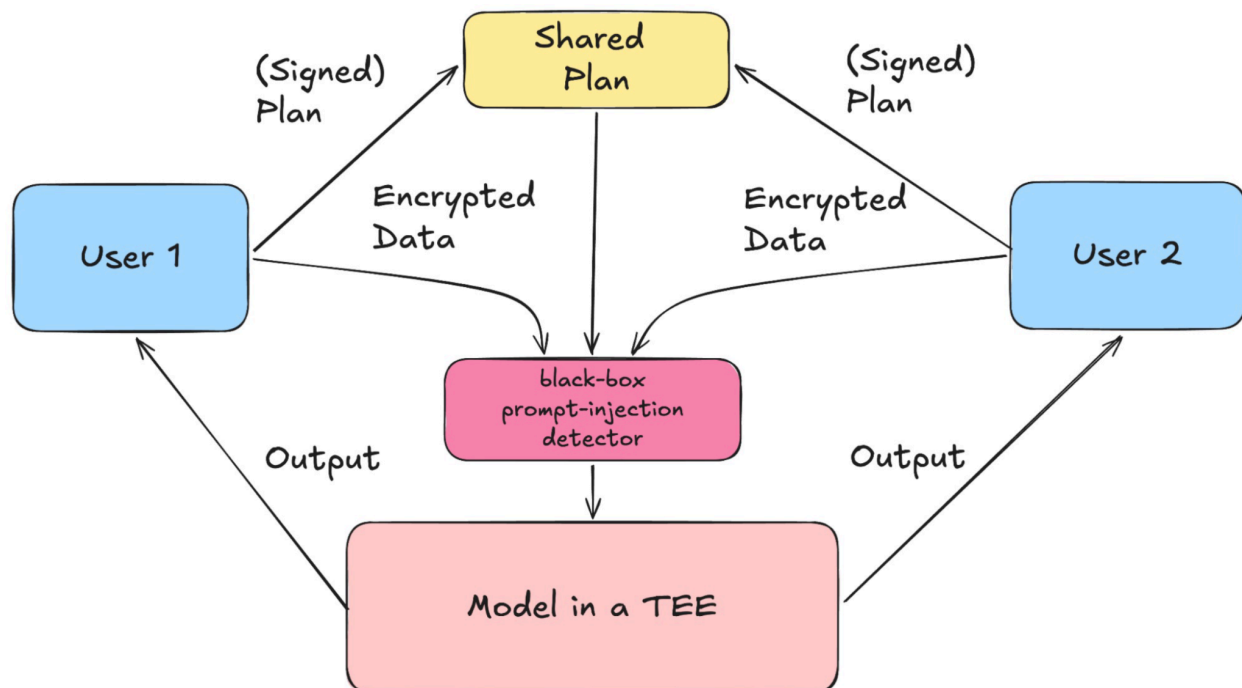
There are a few possibilities to resolve this:

1. **Don't commit to a single model.** What's out-of-distribution for one model usually isn't for another, so the plan can authorize several interchangeable models and pick one at

runtime using trusted randomness. Now a working injection has to break *all* the candidate models at once, which is much harder than attacking a known target.

2. **Each side runs its own black-box filter.** I put my own prompt-injection detector right in front of the model, and you put yours. And because you can't probe mine, it's very hard to craft an injection that gets past it. And in general, you only have to trust your own.¹

And then for any prompt injections you detect, you can handle a false positive from the detector in the same way as in section 3.2 - with the appeals process; and for any confirmed true positive, you respond through some organizational process (e.g., "I'll sue you if you prompt-inject the model.").



4 Applications

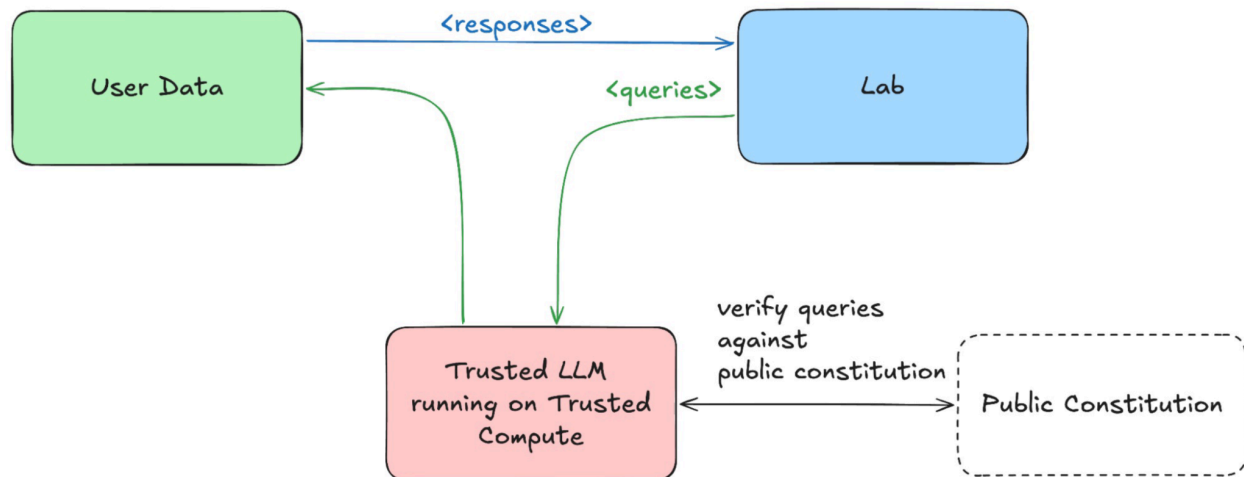
However, we want to now focus on near-term instances which we think are most useful.

4.1 Application: Verifiably Scoped Monitoring (reconciling safety monitoring with zero-data retention)

Frontier AI labs have legitimate reasons to monitor model traffic: detecting misuse of dangerous capabilities, identifying jailbreaks and prompt injection, surfacing harmful usage patterns, and

¹ This is a [panopticon](#)-style defense: in a panopticon, one is unsure if they are being monitored, but is incentivized to act as if they are being monitored because they might be, and the consequences of getting caught are sufficiently high. Here, since a caught injection attempt is costly, the rational move is not to try.

collecting aggregate statistics. The difficulty is that monitoring infrastructure is scope-agnostic. A classifier reading conversations against a natural-language description of what to flag is the *same system* whether that description says "bioweapon synthesis" or "political affiliation" — only the description changes, and the monitored party can't see the description. So assurances about scope must be contractual (not cryptographic) and the customer is asked to trust that the monitor only does what it says.



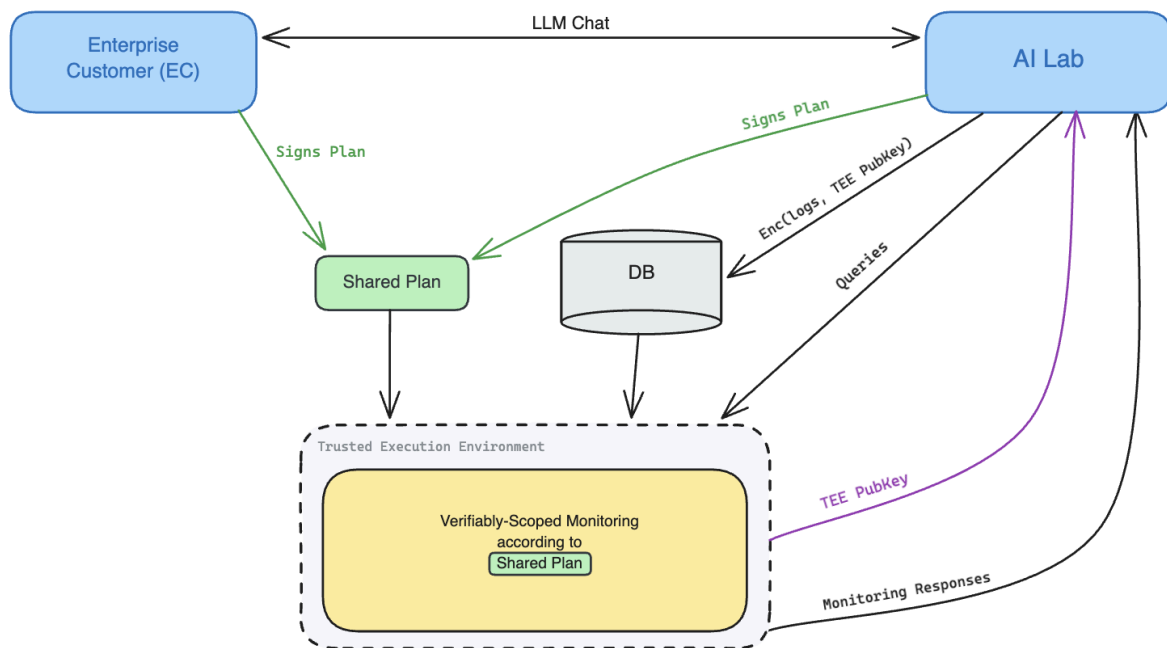
Either the lab keeps the data and does its oversight, and the customer needs to trust the lab entirely, or the lab accepts zero-data-retention (ZDR) terms and loses the visibility it needs. One of the big reasons that labs resist offering ZDR is because under ZDR a lab can only monitor a single query at a time. A filter on one message in isolation can't notice a set of individually-innocuous queries that are jointly dangerous, or a jailbreak assembled gradually across a session. As such, labs generally prefer to only offer ZDR to companies who they trust.

Our proposed fix is to decouple *retention* from *access*: instead of deleting the data, the lab encrypts it under a key only the box (the TEE) holds. Concretely:

1. The customer and the lab agree on and sign a plan fixing the scope of monitoring — which query categories are allowed, what output format is required, any aggregation floor.
2. The box generates an encryption keypair *inside* the enclave and publishes the public key alongside its attestation. The private key never leaves in plaintext; it's sealed to the attested binary on that specific chip, so only a fresh boot of the same verified image can recover it.
3. As conversations happen, the lab encrypts each log under the box's public key and stores only ciphertext. It holds no decryption key and can't read what it stored.
4. At monitoring time, three things go in: the signed plan, the ciphertext logs, and the lab's queries. The box decrypts the logs internally, checks each query against the signed scope, runs the ones that conform, and releases only the agreed outputs. Anything out of scope is rejected and never runs.

The output stage is where you can apply strong guarantees. Because the output filter can be deterministic code, you can force the release to be, say, a single bit — which gives you a worst-case bound: no more than one bit leaves the process, no matter what happens inside.

It's important to be very explicit that this is no longer "Zero" Data Retention, the logs do exist, and through poor security, the logs could be leaked. However, the logs are stored only as ciphertext which is only decryptable inside the co-signed box.



4.2 Application: recurring third-party audit

Any verification scheme needs two kinds of commitment:

- A **positive commitment** — proof that the thing you claim is happening actually is happening. Standing up the pipeline and watching it run gives you this.
- A **negative commitment** — proof that nothing *else* is going on.

When we create a pipeline to evaluate one type of thing (e.g. *did a lab use personal data in a training query?*) we are creating a positive commitment, but by default we don't have a negative commitment that they aren't making the same query elsewhere (unmonitored by the auditor).

However, we already have precedent of a group like [METR sending in an embedded red-teamer](#) into a frontier lab (Anthropic). Given that [METR already sends questionnaires](#) to the labs, the

goal of sending a red-teamer is either for auditors to be able to ask "unknown unknown" questions or reduce the trust assumptions that they have in the people filling out the questionnaires.

Building off of this existing red-teaming auditor infrastructure, we can imagine a similar world where an independent evaluator (e.g. a METR employee) gets access to a lab's data and part of its codebase, figures out which questions are worth asking, and encodes one as a pipeline (e.g. *did you use this personal data in a training query?*, *are you training on the model's chain-of-thought?*) Then they set up the pipeline runs in the box and emit a bounded output (e.g., one bit a week). The auditing agency gets both a positive commitment (the answer to the pipeline) and a negative commitment (because the auditor had access to the whole system).

The critical part of such a scheme is for the auditor to periodically return: each time, the auditor re-checks that the existing pipelines are still wired into production, and goes looking for new questions to be made into pipelines.

5. Limitations and trust assumptions

Any trusted-third-party-in-a-box rests on two underlying assumptions, simply put: that it's a *trusted LLM*, and a *trusted box*. (Shumailov et al. explore these assumptions in [more depth](#).)

The box has to be trustworthy. In our case the box is a TEE, and in some cases TEEs do have side-channel and microarchitectural attacks; so the security of the whole thing is bounded by the current state of confidential-compute hardware.

The LLM has to be both capable and inspectable. It needs to be good enough to act as a faithful classifier, *and* open enough for both sides to examine. There's also a subtler version of this: even a capable model can fail if it lacks the *information* to answer — figuring out, say, how a lab's cluster scheduler allocates compute might need context the box just doesn't have. Some questions are an intelligence problem; others are an access problem.

Importantly, what trust assumptions we are okay with depends on the setting. Two users might be perfectly happy running this in an AWS EC2 instance because they both trust AWS, whereas in a US–China deployment the hardware and software vulnerabilities may need to be thought through much more carefully.

6. A Reference Implementation

We built and released an open-source reference implementation, *auditor-in-a-TEE*, that runs the whole protocol end-to-end on commodity confidential-compute hardware — a Python service with a single-page web client and a CLI, deployed inside a Tinfoil confidential VM ([webapp](#); [source](#)).

PLAN CONTENTS

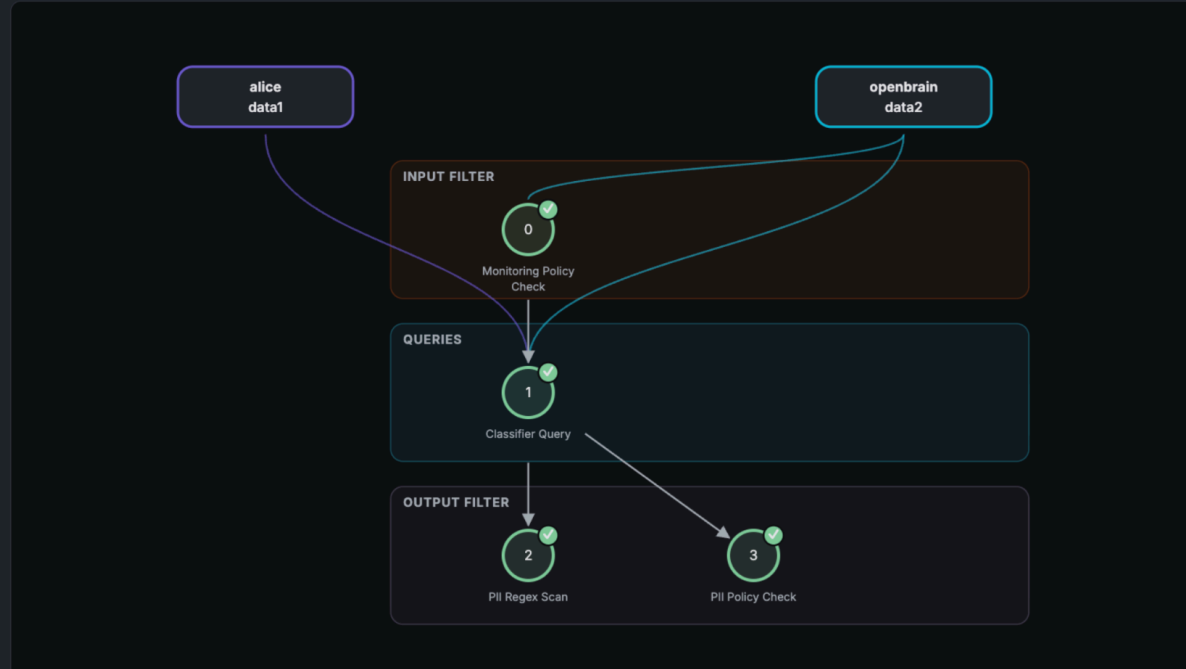
Plan ID: 970e230d

Hash: f2799f87998a3768b67d07b202815dc6f4748b54afd08a55464142c1fe4f4e75

Two parties collaborate on an aggregate-only study of chat-message usage. User 1 uploads their ChatGPT-style messages. User 2 (the researcher) uploads a classifier prompt plus a fixed label set. The TEE first audits the classifier prompt against the published monitoring policy (VALID / INVALID). If approved, it applies the prompt verbatim to each message and returns only aggregate label counts with k-anonymity suppression. A PII Policy Check then verifies the output contains no raw text, identifiers, or small buckets before release.

PIPELINE

Click a user's pill to see their data format, or any step node to see its prompt / code.



INPUT FILTER

PASSED

Monitoring Policy Check

Audits the researcher's submitted classifier spec against the published monitoring policy. Emits VALID or INVALID. Gates whether any query runs.

Step 0 · run_llm · gemma4-31b

QUERIES

PASSED

Classifier Query

Applies the researcher's approved classifier prompt to each user message and returns aggregate label counts. Output is not surfaced to the operator until the PII policy check passes.

Step 1 · run_llm · gemma4-31b

OUTPUT FILTER

PASSED

PII Regex Scan

Deterministic regex pass over the classifier report. Flags emails, phone numbers, SSNs, credit-card digit runs, IPv4 addresses, and any markdown-table cell with a count of 1 or 2 (k-anonymity < 3). Emits VALID or INVALID.

Step 2 · run_python

PII Policy Check

Reviews the classifier output against the privacy constitution. Emits VALID or INVALID. Only on VALID does the operator see the final result.

Step 3 · run_llm · gemma4-31b

We emphasize a few of the points of the webapp and CLI:

- Plans are explicit and signed. Each plan is an ordered list of `run_llm` and `run_python` steps grouped into input, query, and output stages. A `run_llm` step pins a

specific open model and a constitution it must satisfy; a `run_python` step is a snippet both parties sign verbatim.

- **The chain of trust is publicly verifiable.** The deployer repository publishes the exact container image and digest; a Sigstore bundle binds that configuration to the expected binary measurement; the hardware produces a signed attestation at boot; and the measurements are compared on each connection. An external party can verify the running enclave against the public repositories using the Tinfoil CLI (`tinfoil attestation verify`).
- **Events are logged.** An append-only ledger records plan creation, signatures, data submission, per-stage verdicts, and appeals. Raw queries and data do not appear; only outcomes and accepted-appeal summaries.
- **We provide a worked example.** In the `openbrain_audit` plan, a user uploads a batch of her own chat messages, and the lab is allowed exactly one question: what percentage of these conversations are work-related versus personal? The box checks that the question conforms to the constitution, runs the classification, aggregates under a fixed privacy floor (no PII, no raw text, $k = 3$), and releases only the aggregate counts. The lab learns the work-vs-personal split; the user learns that nothing else was pulled out.

7. Current and Future States

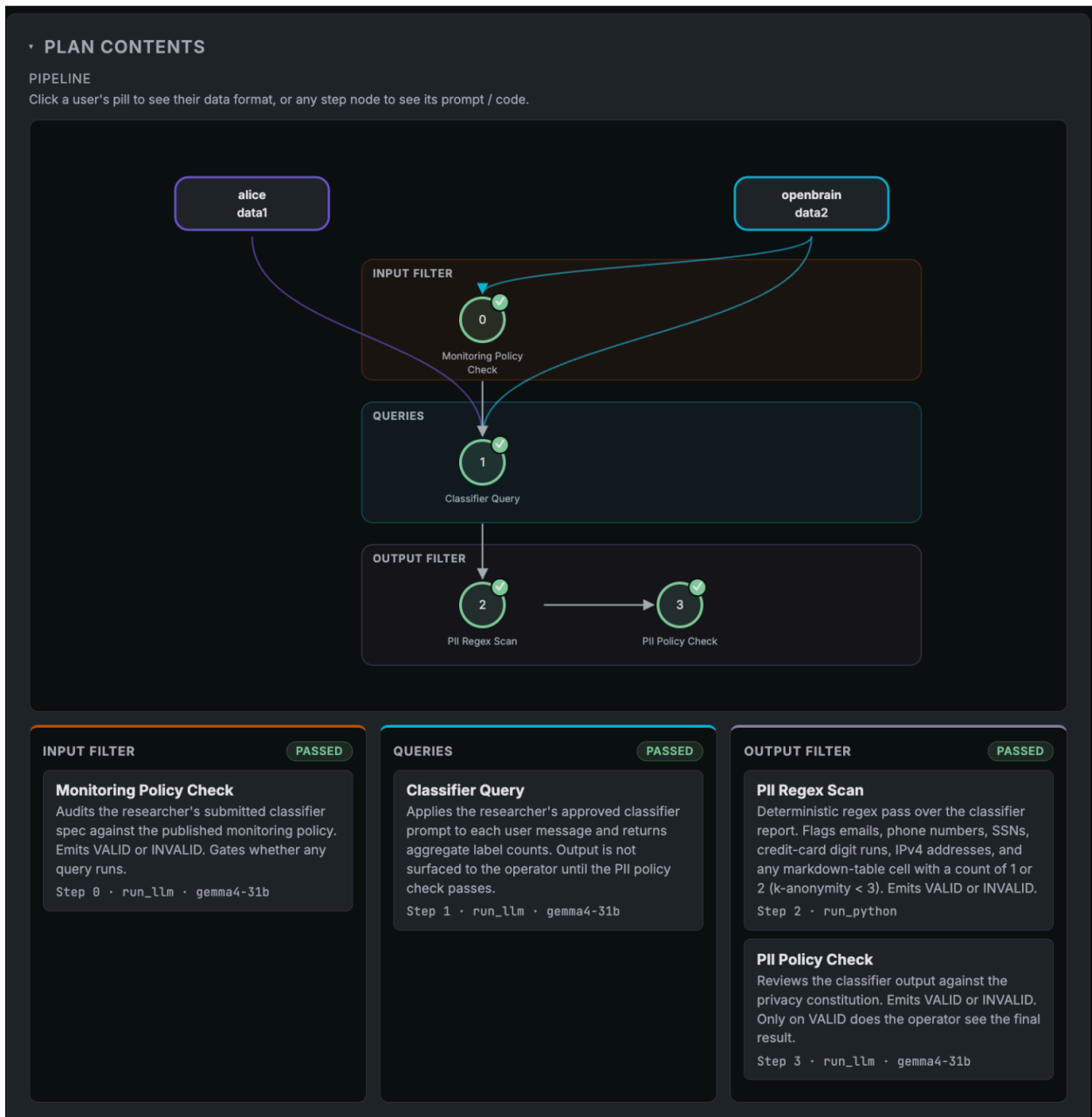
This is a working reference implementation - the protocol runs end-to-end in a real TEE, but has not been stress-tested by a counterparty who actually wants to break them. The most useful thing that could happen next is for people to actually *use* this. The best outcome I can imagine is someone picking up the UI and trying to write a plan for a problem they actually care about.

Appendix

Example Computation Graph for Privacy Preserving Monitoring

Below we show an example plan two parties might agree to as a computation graph. In this case, two parties submit data (alice's chat messages, openbrain's monitoring queries) into a pipeline of signed nodes grouped into three stages. The input filter (node 0) audits the lab's submitted classifier prompt against the published monitoring policy and emits VALID/INVALID (this is the scope check that gates whether anything runs at all). The query stage (node 1)

applies the approved prompts. The output filter (nodes 2–3) runs a deterministic PII regex scan with a k-anonymity floor followed by a policy check, and only on VALID does the lab see the result. Each node is either a run_python step (signed verbatim) or a run_llm step against a named open model (here gemma4-31b); both the graph structure and each node are co-signed.



Ledgers for handling abusing the false positive appeals process

However, the same loop that lets an honest party clear a flag also lets a dishonest one *probe* it — resubmitting slightly different modifications or secondary prompts, fishing for an edit that defeats the classifier. To deal with these potential attacks we propose including a public ledger as part of the appeals process.

When an appeal succeeds, the box writes a short record to a per-plan, append-only, public ledger: an `appeal_accepted` event plus a few word summary of the change, emitted by an LLM inside the enclave (e.g. "Corrected a misspelled name."). The appeals process also records failures by logging every `query_rejected`, `appeal_filed`, and `appeal_rejected` event. So an honest party's ledger shows the occasional one-shot correction, while someone optimizing towards a pass leaves an indelible trail of rejected attempts.